

深入解析 uniApp <input> 组件：从基础到场景应用

你想详细了解 uniApp 中的 `<input>` 组件，我会从基础用法、核心属性、事件、类型适配和注意事项等方面，给你做一个全面且易懂的讲解。

1. input 组件的核心作用

`<input>` 是 uniApp 提供的跨平台输入框组件，替代了 HTML 中的 `<input>` 和 `<textarea>`，适配了 App、H5、各小程序平台，用于接收用户的文本、数字、密码等输入内容，也是表单交互中最核心的组件之一。

2. 基本用法

先看一个最简单的示例，包含基础的双向绑定和输入事件：

Code block

```
1  <template>
2    <view class="input-container">
3      <!-- 基础输入框 -->
4      <input
5        v-model="inputValue"
6        type="text"
7        placeholder="请输入内容"
8        @input="handleInput"
9      />
10     <view>你输入的内容：{{ inputValue }}</view>
11   </view>
12 </template>
13
14 <script>
15 export default {
16   data() {
17     return {
18       inputValue: '' // 绑定输入框值
19     }
20   },
21   methods: {
22     handleInput(e) {
```

```
23      // e.detail.value 是实时输入的内容（和v-model值一致）
24      console.log('实时输入: ', e.detail.value);
25  }
26  }
27  }
28  </script>
29
30  <style scoped>
31    .input-container {
32      padding: 20px;
33    }
34    input {
35      border: 1px solid #eee;
36      padding: 10px;
37      margin-bottom: 10px;
38    }
39  </style>
```

3. 核心属性（按常用程度排序）

属性名	类型	默认值	核心说明
type	String	text	输入框类型，关键 - number：数字输入框 idcard：身份证号输入框 - digit：数字输入框 password：密码（内容隐藏） textarea：多行文本输入框
v-model	String/Number	-	双向绑定输入框的 v-model 是 uniApp 的 糖，本质上是通过 实现的，无需开发 数据与视图的同步 一、v-model uniApp 中 input 的 - 将数据模型中的 性，实现“数据驱动视图” - 监听 input 组件的 时，自动将事件绑定 数据模型中，实现“视图驱动数据”

- 最终达成“数据同步”的双向绑定（等价于手动编写以程序）：

Code block

```
1 <!-- 手
   model)
   :value:
   => inpu
   e.deta
```

二、核心用法

1. 基础绑定（文

直接将 v-model 绑定到文本输入、数字等基础输入元素上，Vue 会自动适配（如输入类型为 Number）：

Code block

```
1 <templ
   入 --><
   model='
   placeho
   - 绑定数
   -><inpu
   type="n
   入年龄"
   /></vi
   <scrip
   {return
   类型age
   数字后仍
   {conve
   Number
   转为数字
```

2. 绑定密码类型

当 input 类型为 password 时，绑定的变量会同步以隐藏字符显示（如密码）。

```
1 <input
  type="password"
  输入密码
```

3. 绑定多行文本

当 input type="text" 用于绑定多行文本内容，支持在模板表达式中引用变量（如：{{variable}}）：

```
Code block
1 <input
  type="text"
  输入备注
```

三、注意事项

- 1. 变量初始值：当使用 v-model 双向绑定初始值时，data 中初始化值必须与双向绑定失效后的值一致；
- 2. 数字输入的类型：当使用 type="number" 时（type="number"），需要确保输入为数字类型（如：123），需要确保输入为数字类型；
- 3. 避免与 value 属性冲突：当使用 v-model 双向绑定 value 属性时，可能会导致冲突，出现异常；
- 4. 跨平台兼容性：v-model 在大部分主流浏览器和框架中均完全兼容，但在某些特殊场景下可能需要手动指定 value + @input 事件；
- 5. 自定义输入框：当使用自定义输入框时（如限制只能输入数字），需要在事件处理函数中手动同步修改后的值；

```
Code block
1 <!-- 限制只能输入数字 -->
  v-model="value"
  @input="handleInput"
  placeholder="请输入数字"
  <script>
    {return
```

			<pre>{filter: (value) => { return value.toLowerCase().replace(/[\u4e00-\u9fa5]/g, ''); }}</pre> 四、特殊场景 uniApp 支持 v-model 的多种特殊处理逻辑： 1. .trim 修饰符 自动过滤输入内容的首尾空格，适用于日期、金额等场景（中间空格保留） Code block <pre>1 <input type="text" v-model="value" :placeholder="placeholder" .trim /></pre> 2. .number 修饰符 自动将输入内容转换为数字，变量值会变为 Number 类型，适用于年龄、金额等纯数字输入 Code block <pre>1 <input type="text" v-model="value" :placeholder="placeholder" .number /></pre>
placeholder	String	-	输入框为空时的提示文字
placeholder-style	String	-	提示文字样式，如 <code>size:14px</code>
disabled	Boolean	false	是否禁用输入框（disabled）
maxlength	Number	140	最大输入长度，默认 140
focus	Boolean	false	是否自动获取焦点（focus）
confirm-type	String	done	确认按钮的文本，默认为 done

			键盘右下角按钮文 done / go / nex 有差异)
confirm-hold	Boolean	false	点击完成按钮后是
password	Boolean	false	等价于 type="pa
clearable	Boolean	false	是否显示清除按钮 空)
adjust-position	Boolean	true	输入时是否自动上

4. 常用事件

事件名	触发时机	回调参数	核心用途
@input	输入内容实时触发	e.detail.value（当前输入值）	实时监听输入、做（如限制字数/格
@confirm	点击键盘右下角“完成/搜索”等按钮时触发	e.detail.value（当前输入值）	输入完成后提交（录）
@focus	输入框获取焦点时触发	e.detail（包含光标位置等）	聚焦时做提示、显
@blur	输入框失去焦点时触发	e.detail.value（当前输入值）	失焦时做校验（如检查）
@clear	点击清除按钮（clearable=true）时触发	-	清空后做额外处理态）

5. 常见场景示例

示例1：手机号输入（带格式校验）

Code block

```
1 <template>
2   <input
3     type="number"
4     v-model="phone"
5     maxlength="11"
6     placeholder="请输入手机号"
7     @blur="checkPhone"
8   />
```

```

9     <view v-if="phoneError" style="color:red;">{{ phoneError }}</view>
10 </template>
11
12 <script>
13 export default {
14   data() {
15     return {
16       phone: '',
17       phoneError: ''
18     }
19   },
20   methods: {
21     checkPhone() {
22       const reg = /^1[3-9]\d{9}$/;
23       if (this.phone && !reg.test(this.phone)) {
24         this.phoneError = '请输入正确的手机号';
25       } else {
26         this.phoneError = '';
27       }
28     }
29   }
30 }
31 </script>

```

示例2：密码输入（带显示/隐藏切换）

Code block

```

1  <template>
2    <view style="position: relative;">
3      <input
4        :type="showPwd ? 'text' : 'password'"
5        v-model="pwd"
6        placeholder="请输入密码"
7      />
8      <button
9        type="default"
10       size="mini"
11       style="position: absolute; right: 10px; top: 5px;"
12       @click="showPwd = !showPwd"
13     >
14       {{ showPwd ? '隐藏' : '显示' }}
15     </button>
16   </view>
17 </template>
18

```

```
19 <script>
20 export default {
21   data() {
22     return {
23       pwd: '',
24       showPwd: false
25     }
26   }
27 }
28 </script>
```

示例3：多行文本输入（textarea 类型）

Code block

```
1 <template>
2   <input
3     type="textarea"
4     v-model="content"
5     placeholder="请输入备注"
6     maxlength="200"
7     @input="countLength"
8   />
9   <view>{{ content.length }}/200</view>
10 </template>
11
12 <script>
13 export default {
14   data() {
15     return {
16       content: ''
17     }
18   },
19   methods: {
20     countLength() {
21       // 实时显示字数统计
22     }
23   }
24 }
25 </script>
```

6. 平台差异与注意事项

1. type 适配：

- H5 端： `type="number"` 仍能输入非数字（可通过 `@input` 过滤），小程序/App 会限制为数字键盘。
- App 端： `adjust-position` 可能因机型不同，软键盘上推效果有差异，可结合 `uni.getSystemInfo()` 做适配。

2. 双向绑定：

- uniApp 的 `v-model` 已封装跨平台兼容，无需手动绑定 `@input` 更新值（但仍可监听 `@input` 做额外处理）。

3. 清除按钮：

- `clearable` 属性在部分小程序（如支付宝）可能不生效，可手动实现清除逻辑。

4. 焦点控制：

- 动态修改 `focus` 为 `true` 可主动聚焦（如点击“编辑”按钮聚焦输入框），聚焦后记得在合适时机设为 `false`。

5. 表单提交：

- 放在 `<form>` 标签内的 `input`，提交时可通过 `form` 的 `@submit` 事件获取所有输入框值（需给 `input` 加 `name` 属性）。

总结

1. `<input>` 是 uniApp 跨平台输入组件，替代 HTML 原生输入框，支持文本、数字、密码等多种输入类型。
2. 核心用法是通过 `v-model` 双向绑定值，结合 `@input` / `@confirm` / `@blur` 处理输入逻辑。
3. 重点关注 `type`（输入类型）、`maxlength`（长度限制）、`clearable`（清除按钮）等属性，以及平台间的适配差异。

如果需要针对某一具体场景（比如“验证码输入框”“金额输入框”）写完整的示例代码，我可以帮你定制。

（注：文档部分内容可能由 AI 生成）